Harbor Watch – Complete Code Explanation

Every term, function and working in harbor_watch_nvdec_split_concat_viewer_socketio.py

This document walks through the file from beginning to end: every imported library, every new technical term, every class and function, how video travels from RTSP to NVDEC, how split_concat produces the panorama, how the Python viewer displays it, how JPEG, Base64 and Socket.IO send it to the UI, and how the whole program executes. The goal is to understand not just what the code does, but why each component exists and where it is used.

1. Overall Architecture

The application has three major stages.

Stage 1: Video acquisition

```
RTSP Camera -> PyNvVideoCodec -> NVDEC -> GPU RGB Frames
```

Stage 2: Video processing

```
GPU RGB Frames
      |
      v
split_concat.run()
      |
      v
  Panorama
      |
      +--------------------+
      |                    |
      v                    v
Python Viewer       Result Generation
                           |
                    JPEG Encoding
                           |
                    Base64 Encoding
                           |
                       Socket.IO
                           |
                        Web UI
```

Stage 3: Two simultaneous outputs

Output	Technology	Purpose
Python Viewer	CUDA + OpenGL	Display panorama locally
Web UI	Socket.IO	Display panorama in the browser

There are no detection models, classification models or analytics in this implementation.

2. Python Libraries and Imports

```
import os, sys, time, base64, threading
from datetime import datetime
```

Library	Definition	Where used
os	Operating System interface	File paths, environment variables
sys	Python interpreter interface	Adding module paths
time	Time-related functions	Timestamps, delays, FPS calculations
base64	Binary-to-text encoding	Converting JPEG to Base64
threading	Multithreading support	Independent camera decoding
datetime	Date and time handling	Imported but not currently used

`os.path.join(os.path.dirname(torch.__file__), "lib")` constructs a path to the PyTorch library directory. `time.perf_counter()` measures how long an operation takes, which is useful for measuring decoding performance.

2.1 NumPy

NumPy is a Python library for numerical computing and multidimensional arrays. An image is essentially a multidimensional array. For example, `image.shape` might return `(2160, 3840, 3)`: height 2160 pixels, width 3840 pixels, 3 channels (RGB or BGR depending on format). In this code NumPy handles compressed packet memory, tensor-to-array conversion, image data manipulation and data type conversion.

2.2 PyTorch

PyTorch is a machine learning and tensor computation framework. No ML models run here, but PyTorch provides CUDA tensor operations: `torch.cuda.is_available()` checks for CUDA, `torch.cuda.Stream()` creates a CUDA stream, `torch.cuda.Event()` creates a synchronization event, and `torch.from_dlpack()` converts a DLPack-compatible object into a tensor. PyTorch is used for GPU computation, not model inference.

2.3 OpenCV

OpenCV is an image and computer vision library. Here it is used for resizing (`cv2.resize(image, (2000, 270))`), color conversion (`cv2.cvtColor(image, cv2.COLOR_RGB2BGR)`), JPEG encoding (`cv2.imencode(".jpg", image)`) and window cleanup. OpenCV performs no object detection in this version.

2.4 PyNvVideoCodec

`import PyNvVideoCodec as nvc` is one of the most important imports. It is NVIDIA's Python interface for GPU-accelerated video decoding and encoding; here it is used for decoding and provides access to NVDEC.

What is NVDEC? NVIDIA Video Decoder, a dedicated hardware decoding engine on supported NVIDIA GPUs.

```
Normal:  RTSP -> CPU Decoding -> CPU Frame -> GPU Transfer
NVDEC:   RTSP -> NVIDIA NVDEC -> GPU Frame
```

This reduces CPU decoding workload. NVDEC supports hardware decoding of compatible codecs such as H.264 and HEVC, depending on the GPU.

3. CUDA and Custom Extension Setup

```
TORCH_LIB = os.path.join(os.path.dirname(torch.__file__), "lib")
```

What is a DLL? Dynamic Link Library: compiled functions that applications load when required, used extensively on Windows. `os.add_dll_directory(TORCH_LIB)` adds a directory to the DLL search path so Windows can find the required PyTorch libraries.

`BASE_DIR = os.path.dirname(os.path.abspath(__file__))` uses three pieces: `__file__` (the current file location), `os.path.abspath()` (converts to an absolute path) and `os.path.dirname()` (extracts the directory).

`FUSED_BACKEND = os.path.join(BASE_DIR, "fused_backend")` points to the custom compiled CUDA backend directory. `sys.path.insert(0, FUSED_BACKEND)` adds it to Python's module search path, because `split_concat` lives there. Then `import split_concat` imports the custom split-and-concatenation implementation.

Important: split_concat is not a standard Python library. It is a project-specific module.

4. CUDA, GPU and CPU

Term	Definition
CPU	Central Processing Unit; general-purpose processor
GPU	Graphics Processing Unit; designed for highly parallel operations
CUDA	NVIDIA platform for executing computations on NVIDIA GPUs

The code uses CUDA because frames are decoded on the GPU and processed by a custom CUDA extension.

What is a CUDA kernel? A function executed on the GPU. split_concat may use CUDA kernels internally for image transformations; instead of processing pixels one by one on the CPU, the GPU processes many pixels in parallel.

5. Socket.IO Initialization

```
import eventlet
eventlet.monkey_patch()
import socketio
```

What is Eventlet? A Python networking library supporting cooperative multitasking, so network operations do not block the entire program unnecessarily.

What is monkey_patch()? Modifying or replacing certain standard-library functions with alternative implementations. Eventlet uses it to make supported operations cooperative.

Implementation note: In this file `eventlet.monkey_patch()` is called after several other libraries are already imported. Eventlet generally recommends applying it very early, before importing modules that use networking or threading. This ordering should be reviewed during integration.

What is Socket.IO? A real-time, bidirectional communication library that lets the Python backend and the frontend exchange messages. The backend sends video frames and the browser receives them.

5.1 Socket configuration

```
SOCKET_HOST = "0.0.0.0"
SOCKET_PORT = 12010
```

A host identifies the network interface the server listens on. `0.0.0.0` means listen on all available IPv4 interfaces. It does not mean the frontend should connect to 0.0.0.0; the frontend must use the actual IP of the server machine. A port identifies a particular network service, for example `http://192.168.100.20:12010` (the IP is an example).

5.2 Creating the Socket.IO server

```
sio = socketio.Server(async_mode="eventlet", cors_allowed_origins="*")
```

Term	Definition
socketio.Server()	Creates a Socket.IO server
async_mode	Specifies the asynchronous networking implementation
cors_allowed_origins	Controls which website origins can connect

What is CORS? Cross-Origin Resource Sharing. If the frontend is at `http://192.168.100.20:3000` and the backend at `http://192.168.100.20:12010`, the different ports make them different origins. CORS controls whether the browser may make cross-origin requests. `"*"` permits all origins; for production, restrict it to your actual frontend origin.

5.3 WSGI application

`socket_app = socketio.WSGIApp(sio)`. WSGI (Web Server Gateway Interface) is a standard interface between Python web applications and web servers. Socket.IO handles communication, WSGI provides the application interface, and Eventlet runs the application.

```
Frontend -> HTTP / WebSocket -> Eventlet Server
         -> WSGI Application -> Socket.IO Server
```

6. Namespaces – /front and /rear

```
@sio.event(namespace="/front")
def connect(sid, environ):
    print(f"[SOCKET] Front UI connected: {sid}")

@sio.event(namespace="/rear")
def connect(sid, environ):
    print(f"[SOCKET] Rear UI connected: {sid}")
```

A namespace is a logical communication channel inside a Socket.IO server. There are two, /front and /rear, so the frontend can subscribe to the front and rear camera streams separately.

```
          SOCKET.IO SERVER
                 |
          +------+------+
          |             |
       /front         /rear
          |             |
       Front UI      Rear UI
```

SID is the Session ID assigned to a connected client within a namespace (e.g. Front UI connected: ABC123).
environ is the WSGI environment dictionary with information about the HTTP request and connection.

What is a decorator? `@sio.event(namespace="/front")` is a decorator: it modifies or registers a function. Here it registers `connect()` as the connection handler for /front. A `disconnect(sid)` function registered the same way runs when the frontend disconnects.

7. Camera Configuration

```
SOURCE_MODE = int(os.getenv("SOURCE_MODE", "1"))
CAMERA_MODE = int(os.getenv("CAMERA_MODE", "1"))
```

An environment variable is a configuration value stored outside the source code, letting you change settings without editing the program.

Value	Meaning
SOURCE_MODE = 1	RTSP streams
SOURCE_MODE = 0	Local video files
CAMERA_MODE = 1	CCD
CAMERA_MODE = 0	IR

CCD (Charge-Coupled Device) is an image sensor technology. IR (Infrared) cameras capture infrared radiation rather than relying only on visible light.

7.1 What is RTSP?

Real-Time Streaming Protocol: a network protocol used to control and access multimedia streaming sessions.
Example: `RTSP_URL_DAY_FRONT = "rtsp://192.168.100.10:554/rtpstream/config1"`

Part	Meaning
rtsp://	Protocol
192.168.100.10	Camera/server IP
554	RTSP port
/rtpstream/config1	Stream path

RTP (Real-time Transport Protocol) carries real-time audio and video data. RTSP controls the session, while RTP commonly carries the media packets.

8. The RTSPReader Class

This is the most important class in the video acquisition section. A class is a blueprint for creating objects; each camera gets its own RTSPReader object (`cam_front = RTSPReader(...)`, `cam_rear = RTSPReader(...)`). Every camera must open its stream, decode video, store frames, reconnect if disconnected and provide frames to the main processing loop.

8.1 Constructor – __init__()

```
def __init__(self, url, name="cam", reconnect_delay=2.0, out_w=None,
             out_h=None, source_mode=1, video_path=None, gpu_id=0):
```

`self` refers to the current object, letting it access its own variables and methods.

Parameter	Definition	Purpose
url	Camera RTSP URL	Connects to camera
name	Camera identifier	Logs and debugging
reconnect_delay	Delay before reconnecting	Handles disconnections
out_w	Output width	Accepted for compatibility; currently ignored
out_h	Output height	Accepted for compatibility; currently ignored
source_mode	Input selection	RTSP or file
video_path	Local video path	Used in file mode
gpu_id	GPU identifier	Selects GPU (0 = first GPU, 1 = second)

8.2 Thread creation

```
self._thread = threading.Thread(target=self._reader_loop, daemon=True,
                                name=f"reader-{name}")
self._thread.start()
```

Multithreading means executing multiple threads within the same program, so the four camera readers decode independently. A daemon thread is a background thread that does not prevent the process from exiting. `target` specifies the function the thread runs (`_reader_loop`), and `start()` begins decoding independently of the main program.

```
            MAIN PROGRAM
                 |
   +-------+-----+-----+-------+
   |       |           |       |
CCD Front CCD Rear  IR Front IR Rear
```

9. The Frame Buffer

```
self._cond = threading.Condition()
self._latest = None
self._latest_ts = 0.0
self._seq = 0
```

Variable	Meaning
_cond	Thread synchronization condition
_latest	Latest decoded GPU frame
_latest_ts	Timestamp of the latest frame
_seq	Frame sequence number

A condition variable lets threads wait until a condition becomes true: the main processing thread waits until a new frame arrives, and the decoder thread notifies it when a frame is ready.

```
DECODER THREAD: Decode Frame -> Store Latest -> Notify Condition
MAIN THREAD:    Wakes -> Process Frame
```

Every newly decoded frame increments `self._seq += 1` (Frame 1 gives seq 1, Frame 2 gives seq 2, and so on), which helps determine whether a new frame has arrived.

Why latest-frame storage? If the camera produces 30 FPS but the pipeline handles 20 FPS, a traditional queue accumulates old frames. A latest-frame buffer replaces old unprocessed frames with the newest one, reducing latency.

10. Statistics Functions

Several performance-monitoring functions help identify decoding bottlenecks.

- `_reset_stats(now)` resets performance counters.

- `_avg_ms(total)` calculates average processing time in milliseconds: average time = total time / number of frames, times 1000.
- `_maybe_report_stats()` prints statistics every 10 seconds.

```
[ccd_front][stats]
fps=29.5 decoded=295 packets=300 decode_errors=0
avg_decode=1.25ms avg_gpu_copy=0.45ms avg_lock_wait=0.02ms
```

FPS (Frames Per Second) is how many frames are processed or decoded per second. Latency is the delay between receiving a frame and displaying it; lower is better for real-time surveillance. A demux error occurs while extracting packets from the stream; a decode error occurs while decoding compressed data. GPU copy is the time spent copying the decoded surface into a separate GPU tensor.

11. _open() – Opening the Camera

This function opens the input source and initializes the NVDEC decoder.

```
src = self.url if self.source_mode == 1 else self.video_path
demux = nvc.CreateDemuxer(src)
```

A demuxer separates the multimedia stream into individual packets (H.264 Video Stream -> Demuxer -> Compressed Packets).

```
dec = nvc.CreateDecoder(
    gpuid=self.gpu_id,
    codec=demux.GetNvCodecId(),
    cudacontext=0,
    cudastream=0,
    usedevicememory=True,
    outputColorType=nvc.OutputColorType.RGB
)
```

Parameter	Definition
gpuid	GPU to use
codec	Video compression format
cudacontext	CUDA execution context
cudastream	CUDA stream identifier
usedevicememory=True	Keeps decoded output in GPU memory
outputColorType=RGB	Requests RGB output

A codec (Coder-Decoder) compresses and decompresses video; examples are H.264, H.265/HEVC and AV1. The codec must be supported by your GPU and the installed NVIDIA decoder library. RGB means Red, Green, Blue, the common image channel representation the decoder is configured to output.

12. _run() – Video Decoding

```
def _run(self, demux, dec, fps):
```

This is the actual decoding loop, continuously reading and decoding packets.

Step 1: Frame period. `period = 1.0 / fps`. At 30 FPS the period is 1/30 = 0.0333 seconds, the interval between frames. In the current file the file-playback pacing code is commented out, so local video playback may run faster than real time.

Step 2: Packet iterator. `it = iter(demux)`. An iterator returns elements one at a time; here, compressed video packets.

Step 3: Retrieve packets. `packet = next(it)` reads the next packet.

Step 4: Decode using NVDEC. `surfaces = dec.Decode(packet)` is the most important line: the compressed packet goes to NVIDIA's decoder, which produces decoded image surfaces. A surface is a memory region containing image data, here a decoded GPU frame.

Step 5: Convert to PyTorch tensor. `frame = torch.from_dlpack(surf).clone()`

- DLPack is a standard that lets different frameworks share tensor memory, so PyTorch can access the decoded surface.
- `clone()` creates a separate copy. The decoder may reuse its original memory for later frames, so copying keeps the frame valid while it is processed.

Step 6: Publish the frame.

```
with self._cond:
    self._latest = frame
    self._latest_ts = time.monotonic()
    self._seq += 1
    self._cond.notify_all()
```

`time.monotonic()` returns a clock value suited to measuring elapsed time, and `notify_all()` wakes threads waiting on the condition.

13. Reconnection

`STALL_TIMEOUT = 15.0`. A stream stall happens when video stops arriving:

```
Camera working -> Frames arriving -> Network interruption
-> No frames arriving -> Stall detected -> Reconnect
```

The code checks `if not is_file and time.time() - last_packet_time > self.STALL_TIMEOUT:`. If no packet arrives for 15 seconds, it reconnects. `reconnect_delay=2.0` waits two seconds before the next attempt, preventing continuous reconnection attempts.

14. Consumer Functions

get_latest() returns `(self._latest, self._latest_ts, self._seq)`: the latest GPU frame, its timestamp and its sequence number.

wait_new(last_seq, timeout=0.2) waits until a new frame arrives. Example: `front_frame, timestamp, seq = cam_front.wait_new(last_seq=10, timeout=0.05)` returns when the sequence becomes 11. This avoids busy spinning, where a program repeatedly checks for something without waiting and consumes CPU unnecessarily.

get_latest_bgr() converts the GPU RGB tensor into a CPU NumPy BGR image. It is not used in the main pipeline and is retained as a compatibility helper.

stop() runs `self._stop_event.set()` then `self._thread.join(timeout=5)`. A threading Event is a synchronization object used to signal another thread; here `_stop_event` tells the decoding thread to terminate. `join()` waits for the thread to finish.

15. Result Generation

`processed_gpu_frame_to_jpeg_base64(gpu_frame)` converts the processed panorama (the output of `split_concat.run(frame)`) into a Base64-encoded JPEG string the frontend can display.

```
GPU Panorama -> PyTorch Tensor -> CPU NumPy Array -> RGB Image
-> Resize -> BGR Conversion -> JPEG Encoding -> Base64 -> Text String
```

15.1 What is a tensor?

A multidimensional array, for example `(H, W, C)` for height, width and channels. The panorama might have shape `(270, 2000, 3)`; the actual shape depends on split_concat.

15.2 detach()

`tensor = gpu_frame.detach()` creates a tensor disconnected from PyTorch's gradient-tracking graph. Autograd is PyTorch's automatic differentiation system used for training; gradient tracking is not needed for video display.

15.3 GPU-to-CPU transfer

`tensor = tensor.to("cpu", non_blocking=False)` transfers the panorama from GPU to CPU memory (D2H, Device-to-Host). This is needed because the current JPEG encoding uses OpenCV on CPU memory.

15.4 NumPy conversion

`array = tensor.numpy()` converts the CPU tensor into a NumPy array.

15.5 HWC and CHW

The code supports two layouts: HWC (Height x Width x Channels, e.g. 270 x 2000 x 3) and CHW (Channels x Height x Width, e.g. 3 x 270 x 2000). Different GPU operations and libraries may produce different layouts, so the code checks the channel dimension and converts CHW to HWC when necessary.

15.6 Data type conversion

`if image_rgb.dtype != np.uint8:` uint8 is an unsigned 8-bit integer representing values 0 to 255, the common pixel format. The code also handles floating-point images (0.0 = black, 1.0 = white) by scaling normalized values into the 0-255 range.

15.7 Image resizing

```
image_rgb = cv2.resize(image_rgb, (UI_WIDTH, UI_HEIGHT),
                       interpolation=cv2.INTER_AREA)
```

With `UI_WIDTH = 2000` and `UI_HEIGHT = 270`, the UI copy is resized to 2000 x 270 to stay compatible with the existing frontend dimensions. Interpolation determines how pixel values are calculated when resizing; INTER_AREA is generally suitable for shrinking. **Important:** this resize affects the Socket.IO output, not the original panorama shown by the Python viewer.

15.8 RGB-to-BGR conversion

`image_bgr = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)`. OpenCV's conventional ordering is BGR while the NVDEC decoder outputs RGB, so conversion is needed before OpenCV's JPEG encoder.

16. JPEG Encoding

```
ok, encoded = cv2.imencode(".jpg", image_bgr,
    [int(cv2.IMWRITE_JPEG_QUALITY), JPEG_QUALITY])
```

JPEG (Joint Photographic Experts Group) is a compressed image format. A raw 2000 x 270 RGB image is 2000 x 270 x 3 = 1,620,000 bytes (about 1.62 MB); JPEG reduces what must be transmitted. `JPEG_QUALITY = 75` controls compression:

Quality	Result
95	High quality, larger file
85	Good quality
75	Balanced quality and size
50	More compression
25	Lower image quality

`ok` indicates whether encoding succeeded; if false, the code raises an error.

17. Base64 Encoding

```
return base64.b64encode(encoded.tobytes()).decode("ascii")
```

Base64 converts binary data to a text representation. Socket.IO can carry binary payloads directly, but the existing frontend protocol expects an image string, so Base64 is used (JPEG binary -> Base64 encoder -> "/9j/4AAQSkZJRgABAQ..."). ASCII is a character encoding that Base64 output fits within. `b64encode()` returns bytes, and `.decode("ascii")` converts them into a Python string.

18. Payload Generation

```
def make_ui_payload(image_base64, frame_id, panoramic_view_id):
    return {
        "message_id": frame_id,
        "sensor_id": 1,
        "panoramic_view_id": panoramic_view_id,
        "frame_id": frame_id,
        "aice_epoch": int(time.time() * 1_000_000),
        "no_of_targets": 0,
        "targets": [],
        "image": image_base64
    }
```

A payload is the data sent along with a communication event.

Field	Meaning
message_id	Unique identifier for the message
sensor_id	Identifies the sensor
panoramic_view_id	Identifies the panorama view
frame_id	Identifies the frame
aice_epoch	Timestamp in microseconds
no_of_targets	Number of detected targets
targets	Detection information
image	Base64 JPEG image

Since detection is disabled, `no_of_targets` is 0 and `targets` is empty; they are kept for compatibility with the existing UI. Epoch time is the time elapsed since January 1, 1970 UTC; the code multiplies seconds by one million to get microseconds.

19. Camera Selection

`get_active_camera_pair(cameras)` returns `cameras["ccd_front"], cameras["ccd_rear"]` when CAMERA_MODE is 1 (CCD), and `cameras["ir_front"], cameras["ir_rear"]` when it is 0 (IR). A separate function means the main loop does not repeatedly check which cameras to use; it simply calls `cam_front, cam_rear = get_active_camera_pair(cameras)`.

20. Main Processing Loop – run_video_pipeline()

`run_video_pipeline(cameras, viewer)` is the central function. It retrieves frames, processes them with split_concat, displays them in Python and sends them through Socket.IO.

20.1 CUDA streams

```
split_stream_front = torch.cuda.Stream()
split_stream_rear = torch.cuda.Stream()
```

A CUDA stream is an ordered sequence of GPU operations. Two streams (front and rear) let the GPU schedule work for the cameras independently. It does not guarantee both run physically at the same time; that depends on GPU resources and scheduling.

20.2 CUDA events

```
front_ready = torch.cuda.Event()
rear_ready = torch.cuda.Event()
front_ready.record(split_stream_front)
front_ready.synchronize()
```

A CUDA event marks GPU operation completion. `record()` places the event after the front processing; `synchronize()` waits until that work completes. The processed panorama must not be read on the CPU before the GPU finishes producing it.

20.3 Frame retrieval

```
front_frame, front_ts, front_seq = cam_front.wait_new(last_front_seq, timeout=0.05)
rear_frame, rear_ts, rear_seq = cam_rear.get_latest()
```

The front camera acts as the trigger; the rear camera supplies its latest available frame at that moment. This is a latest-frame pairing strategy, not strict timestamp synchronization, so front and rear panoramas may correspond to slightly different moments.

21. record_stream()

```
front_frame.record_stream(split_stream_front)
rear_frame.record_stream(split_stream_rear)
```

PyTorch uses a caching memory allocator, and GPU memory can be reused once tensors are no longer needed. When a tensor is used on a different CUDA stream, PyTorch must know that stream still uses the memory. `record_stream()` informs the allocator, preventing premature reuse. It does not synchronize the streams.

22. Split and Concatenation

```
with torch.cuda.stream(split_stream_front):
    last_front_output = split_concat.run(front_frame)
    front_ready.record(split_stream_front)

with torch.cuda.stream(split_stream_rear):
    last_rear_output = split_concat.run(rear_frame)
    rear_ready.record(split_stream_rear)
```

Split divides an image into smaller regions. **Concatenation** combines regions or arrays into a larger image. A **panorama** is a wide image created by combining multiple regions or camera views; in Harbor Watch, split_concat produces the required panoramic output.

```
Original Frame -> [ Region 1 | Region 2 ] -> Combined wide image
```

Important: the exact splitting, cropping, alignment and concatenation logic lives inside the split_concat module, not this Python file. That module must be inspected to explain its internal operations.

23. Python Viewer – OpenGL

`from cuda_gl_viewer import CudaGLViewer`. OpenGL (Open Graphics Library) is a graphics API for rendering images and other visual content. CUDA-OpenGL interoperability lets CUDA and OpenGL share GPU resources, reducing unnecessary CPU/GPU transfers. The viewer displays the processed panorama locally.

```
viewer = CudaGLViewer(window_width=1600, window_height=900,
    title="Harbor Watch - Split/Concat Front + Rear")

keep_running = viewer.show(last_front_output, last_rear_output,
    ready_top=front_ready, ready_bottom=rear_ready)
```

This creates a 1600 x 900 window and shows the front panorama on top and the rear panorama below. The viewer lives on the main thread because many OpenGL windowing systems require window creation and event processing there.

24. Socket.IO Frame Transmission

```
front_b64 = processed_gpu_frame_to_jpeg_base64(last_front_output)
front_payload = make_ui_payload(front_b64, frame_id, 1)
sio.emit("frame", front_payload, namespace="/front")
sio.emit("frame", rear_payload, namespace="/rear")
```

`emit()` sends an event to connected clients. Its parameters are the event name `"frame"`, the payload (image and metadata) and the destination `namespace`. An event is a named message such as `frame`, `connect` or `disconnect`.

The frontend can listen like this (illustrative; your actual frontend should use its existing handling):

```
const socket = io("http://YOUR_SERVER_IP:12010/front");
socket.on("frame", (data) => {
```

```
    const image = document.getElementById("video");
    image.src = "" + data.image;
});
const rearSocket = io("http://YOUR_SERVER_IP:12010/rear");
```

25. Eventlet Sleep

The main loop ends with `eventlet.sleep(0)`, which yields execution to other Eventlet green threads so the Socket.IO server can process incoming connections and outgoing events. A green thread is a lightweight task managed cooperatively by a runtime rather than scheduled as a separate OS thread.

26. Starting the Socket.IO Server

```
def start_socket_server():
    eventlet.wsgi.server(
        eventlet.listen((SOCKET_HOST, SOCKET_PORT)),
        socket_app,
        log_output=False
    )
```

`eventlet.listen(("0.0.0.0", 12010))` creates a listening socket and opens port 12010. `eventlet.wsgi.server()` starts the WSGI server, accepting requests and passing them to the Socket.IO application. `log_output=False` disables standard access logging.

27. main() – How the Application Starts

1. **CUDA check:** `if not torch.cuda.is_available(): raise RuntimeError(...)` stops the application if CUDA is missing.
2. **Containers:** `cameras = {}` and `viewer = None`. A dictionary stores key-value pairs, e.g. `{"ccd_front": camera1, "ccd_rear": camera2, "ir_front": camera3, "ir_rear": camera4}`.
3. **Configuration:** `camera_configs` maps each of the four names to `(url, video_path)`.
4. **Reader creation:** `for name, (url, video_path) in camera_configs.items(): cameras[name] = RTSPReader(...)` creates an independent NVDEC reader per camera.
5. **Initial frames:** `camera.wait_new(last_seq=0, timeout=30.0)` waits for the first decoded frame; after 30 seconds without one, the application raises an error.
6. **Server:** `eventlet.spawn(start_socket_server)` starts a green thread so the Socket.IO server runs without blocking the main video loop.
7. **Viewer:** `viewer = CudaGLViewer(...)` creates the local OpenGL window.
8. **Pipeline:** `run_video_pipeline(cameras, viewer)` starts the complete processing pipeline.

28. Shutdown and Cleanup

A `finally:` block runs whether the program ends normally or with an exception. It calls `viewer.close()`, stops camera threads with `camera.stop()` and closes OpenCV windows with `cv2.destroyAllWindows()`, so camera readers and GPU resources are not left running after exit.

29. Glossary

Term	Full form / Definition
RTSP	Real-Time Streaming Protocol
RTP	Real-time Transport Protocol
NVDEC	NVIDIA Video Decoder
GPU / CPU	Graphics / Central Processing Unit
CUDA	NVIDIA GPU computing platform
CUDA Kernel	Function executed on GPU

Term	Full form / Definition
CUDA Stream	Ordered sequence of GPU operations
CUDA Event	GPU operation completion marker
PyAV	Python interface to FFmpeg
PyNvVideoCodec	NVIDIA Python video codec library
Demuxer	Separates multimedia data into packets
Codec	Compression/decompression technology
H.264	Video compression standard
Surface	Memory containing decoded image data
DLPack	Standard for sharing tensor memory
Tensor	Multidimensional data structure
Clone	Creates a separate tensor copy
NumPy	Numerical computing library
OpenCV	Computer vision and image processing library
RGB / BGR	Red-Green-Blue / Blue-Green-Red channel order
HWC / CHW	Height-Width-Channels / Channels-Height-Width
uint8	Unsigned 8-bit integer
D2H / H2D	Device-to-Host / Host-to-Device transfer
Pinned Memory	Page-locked CPU memory
Thread	Independent execution path
Daemon Thread	Background thread that does not prevent process exit
Iterator	Object that returns values sequentially
Condition Variable	Thread synchronization mechanism
Event	Thread signaling mechanism
Sequence Number	Identifier for a frame's position
FPS	Frames Per Second
Latency	Delay in processing or displaying a frame
Stall	Temporary interruption of incoming data
Reconnection	Re-establishing a lost connection
JPEG	Joint Photographic Experts Group image format
Base64	Binary-to-text encoding
Payload	Data transmitted through an event
Socket.IO	Real-time bidirectional communication
Namespace	Logical communication channel
SID	Session ID
CORS	Cross-Origin Resource Sharing
WSGI	Web Server Gateway Interface
Eventlet	Cooperative networking library
Green Thread	Lightweight cooperative execution task
OpenGL	Open Graphics Library
Interoperability	Ability of two systems to share resources
Panorama	Wide image formed from multiple image regions
Interpolation	Method of calculating pixel values during resizing

Term	Full form / Definition
Epoch Time	Time measured from January 1, 1970 UTC
Environment Variable	External application configuration
DLL	Dynamic Link Library

30. Important Observations About the Current Code

Observation	Explanation
make_packet() is defined but unused	The decoder loop passes demuxed packets directly to Decode().
datetime is imported but unused	Can be removed.
File playback pacing is commented out	Local video files may play faster than their recorded FPS.
Front camera controls the processing loop	A new front frame triggers processing of both panoramas.
Rear frame uses get_latest()	The rear frame may be repeated or slightly out of sync with the front.
JPEG encoding uses CPU	The GPU panorama is transferred to CPU before OpenCV encoding.
get_latest_bgr() is unused	Retained as a compatibility helper.
Four decoder threads are created	Even though only two cameras are active in a given camera mode.

Three recommended improvements

1. **Synchronize front and rear frames.** Currently the front frame triggers processing and the latest rear frame is used. For accurate synchronization, compare timestamps and apply a pairing policy.
2. **Add file playback pacing.** The existing pacing code is commented out; enable it if local video files should play at their original FPS.
3. **Optimize JPEG encoding.** Currently GPU Panorama -> CPU Transfer -> OpenCV JPEG -> Base64, which can bottleneck at high resolutions or FPS.

31. Final Execution Flow

```
MAIN PROGRAM
  -> Initialize CUDA
  -> Initialize Cameras
  -> Start NVDEC Threads
  -> Receive RTSP Data
  -> Demux Video Packets
  -> NVDEC Decoding
  -> GPU RGB Frame
  -> Frame Buffer
  -> Get Latest Front/Rear
  -> SPLIT + CONCAT
  -> PANORAMA
       |-> PYTHON VIEWER -> OpenGL
       |-> RESULT GENERATION -> JPEG -> Base64 -> JSON Payload
                              -> Socket.IO -> /front -> FRONT UI
                                           -> /rear  -> REAR UI
```

The key idea: NVDEC decodes the video, split_concat generates the panorama, OpenGL displays it locally, and Socket.IO transmits the same processed panorama to the browser. No AI inference is involved.

Next Priority

The most important external dependency to understand is split_concat. The main file only calls `split_concat.run(front_frame)` and `split_concat.run(rear_frame)`; the actual image-processing algorithm is inside that module. Understanding its CUDA implementation will explain exactly how the original camera frames become the final panorama.